

INFORME TÉCNICO FINAL

Gestión de Defectos y Validación Final

Post Validator - Proyecto Integrador TYVS

Integrantes

Natalia Rodríguez
Sergio Luis López
José Haider Barreto

Testing y Validación de Software
Maestría en Ingeniería de Software
Universidad de La Sabana - 2026

1. Introducción

Post Validator es una API REST desarrollada en TypeScript con Node.js y Express, que implementa la lógica de negocio de una plataforma de publicación de contenido inspirada en el CMS Ghost. El sistema permite crear, programar, filtrar y controlar el acceso a publicaciones según el tipo de usuario, siguiendo los principios de Arquitectura Limpia con cuatro capas bien definidas: Dominio, Aplicación, Infraestructura y Delivery REST.

Este informe sintetiza los hallazgos del proceso de aseguramiento de calidad aplicado al sistema a lo largo de las Unidades 3, 4 y 5 del curso, cubriendo pruebas unitarias con TDD, pruebas de integración y sistema, y pruebas de carga y rendimiento.

2. Resultados del Proceso de Pruebas

2.1 Resumen ejecutivo

Métrica	Valor	Estado
Tests unitarios (U3)	38 pruebas - 5 servicios de dominio	✅ Completado
Tests integración y sistema (U4)	27 pruebas - integración real, mocks y HTTP	✅ Completado
Escenarios de rendimiento (U5)	6 escenarios con k6	✅ Completado
Total de tests automatizados	65 pruebas	✅ 100% pasando
Cobertura global (U3 + U4)	88% statements	✅ Supera 80%
Cobertura dominio (U3)	99% statements	✅ Supera 80%
SLOs de rendimiento cumplidos	6 de 6 escenarios	✅ Todos cumplen
Defectos encontrados	7 defectos (1 U3, 3 U4, 3 U5)	—
Defectos resueltos	1 (DEF-U3-001)	✅ Resuelto

2.2 Efectividad del TDD en la detección de defectos

El proceso de Desarrollo Dirigido por Pruebas (TDD) aplicado en U3 demostró ser altamente efectivo para detectar defectos en las reglas de negocio del dominio. El ciclo Red-Green-Refactor garantizó que cada regla de negocio fuera validada antes de ser implementada, lo que redujo el riesgo de errores en la lógica central del sistema.

El único defecto encontrado en U3 (DEF-U3-001) fue detectado directamente por el TDD: la implementación inicial de la búsqueda por título utilizaba `String.includes()` de forma case-sensitive, lo que hacía que buscar "typescript" no encontrara un post titulado "TypeScript Avanzado". El test falló en la fase RED y guió la corrección inmediata mediante `toLowerCase()` en ambos lados de la comparación.

Esto ilustra una ventaja clave del TDD: los defectos se detectan en el mismo momento en que se diseña la funcionalidad, antes de que el código llegue a capas superiores o al sistema de producción, reduciendo significativamente el costo de corrección.

2.3 Tipos de defectos por nivel de prueba

Cada nivel de prueba reveló un tipo diferente de defecto, confirmando la importancia de una estrategia de pruebas en pirámide:

Nivel	Defectos encontrados	Tipo de defecto
Unitarias (U3)	1 defecto (DEF-U3-001)	Lógica de negocio incorrecta en dominio
Integración y sistema (U4)	3 defectos (DEF-U4-001/002/003)	Contrato HTTP y validación de entrada
Carga y rendimiento (U5)	3 defectos (DEF-U5-001/002/003)	Limitaciones arquitecturales bajo carga

Las pruebas unitarias detectaron errores en las reglas de negocio puras. Las pruebas de integración y sistema revelaron defectos en el contrato HTTP que las pruebas unitarias no podían ver, como la falta de validación del header x-user-type o la inconsistencia en los mensajes de error. Las pruebas de carga expusieron limitaciones estructurales del repositorio en memoria que solo son visibles bajo volumen sostenido de datos.

2.4 Resultados de cobertura de código

Capa	Statements	Branches	Functions	Lines
Domain Model	100%	100%	100%	100%
Domain Services	98.52%	89.13%	100%	98.38%
Application	89.28%	63.63%	100%	88.46%
Infrastructure	84.61%	100%	83.33%	84.61%
Delivery / REST	65.38%	54.54%	62.5%	66.66%
Global (U3 + U4)	88%	80.41%	89.74%	87.95%

La capa de Delivery tiene la cobertura más baja (65%) porque app.ts, el punto de entrada del servidor, no puede ser cubierto por las pruebas automatizadas. Las pruebas de Supertest crean su propia instancia de Express directamente, sin pasar por el bootstrap del servidor. Esta es una limitación conocida y esperada, no un indicador de baja calidad.

2.5 Resultados de rendimiento

Escenario	VUs máx	p95 (ms)	RPS	Error rate	SLO
Baseline	1	5.34	1.98	0.00%	✓ Cumple
Load	50	57.17	79.77	0.00%	✓ Cumple
Stress	600	11.18	558.19	0.00%	✓ Cumple
Spike	500	26.29	609.86	0.00%	✓ Cumple
Soak	30	33.76	27.71	0.00%	✓ Cumple
Regression	10	23.04	38.54	0.00%	✓ Cumple

El sistema superó todos los SLOs definidos en los 6 escenarios de rendimiento. El p95 más alto fue 57.17 ms bajo carga normal de 50 usuarios virtuales, lo que representa un margen de 3.5x por debajo del umbral de 200 ms. El throughput máximo alcanzado fue 609.86 RPS en el escenario Spike con 500 VUs concurrentes, sin registrar ningún error en ningún escenario.

3. Análisis de Calidad del Sistema

3.1 Módulos más críticos

Basándose en la matriz de trazabilidad del proyecto, los módulos con mayor riesgo operacional son:

- PostValidator: es la puerta de entrada a toda operación de escritura. Un fallo aquí permite datos corruptos en el repositorio.
- AccessControlService: un error en el control de acceso podría exponer contenido premium a usuarios no autorizados, representando un riesgo de negocio significativo.
- PostScheduler: fallos en la validación de fechas podrían resultar en publicaciones en momentos no deseados.
- InMemoryPostRepository: al ser la única capa de persistencia, su fallo implica pérdida total de datos.

3.2 Cumplimiento de SLOs bajo carga normal

El sistema cumple holgadamente con los SLOs definidos bajo carga normal (50 VUs, escenario Load): p95 de 57.17 ms frente a un límite de 200 ms, y 0% de tasa de error. El sistema está preparado para soportar la carga esperada en un entorno académico o de desarrollo.

3.3 Limitaciones arquitecturales identificadas

Las pruebas de carga revelaron limitaciones del repositorio en memoria que no son visibles en pruebas unitarias ni de integración:

- Sin política de evicción: el Map crece indefinidamente con cada petición POST, acumulando más de 275,000 registros durante la batería completa de pruebas. Bajo carga muy alta o ejecuciones prolongadas, esto podría provocar un error de Out of Memory (OOM) en Node.js.
- Degradación progresiva: la latencia muestra una correlación con el volumen de datos acumulados. El Baseline con 0 posts registró p95 de 5.34 ms, mientras que el Regression con 275,000 posts acumulados registró p95 de 23.04 ms, una diferencia de 4.3x atribuible al crecimiento del estado en memoria.
- Contaminación entre escenarios: al no reiniciar el servidor entre pruebas, el estado acumulado de un escenario afecta los resultados del siguiente, lo que dificulta comparaciones válidas entre escenarios.

4. Oportunidades de Mejora

4.1 Mejoras técnicas en el sistema

#	Propuesta	Impacto	Esfuerzo
1	Validar el header x-user-type contra el enum UserType en runtime	Seguridad	Bajo
2	Agregar mensajes descriptivos a los errores de validación HTTP	Experiencia del desarrollador	Bajo
3	Agregar middleware global para errores de JSON malformado	Consistencia del contrato HTTP	Bajo
4	Implementar paginación en GET /api/posts (offset/limit)	Rendimiento con alto volumen	Bajo
5	Implementar política de evicción o TTL en el repositorio	Estabilidad en ejecuciones largas	Medio
6	Migrar a base de datos persistente (SQLite o PostgreSQL)	Persistencia y escalabilidad	Alto
7	Agregar rate limiting con express-rate-limit	Protección ante picos de tráfico	Muy bajo

4.2 Mejoras al proceso de pruebas

- Agregar pruebas E2E con Playwright para validar flujos completos de usuario en una eventual interfaz gráfica.
- Implementar pruebas de contrato (contract testing) con Pact para garantizar compatibilidad entre el cliente y la API.
- Reiniciar el servidor entre escenarios de carga para garantizar comparaciones válidas y aislar el comportamiento de cada escenario.
- Ejecutar el escenario Soak con duración real de 4-8 horas en un entorno dedicado para detectar fugas de memoria reales.
- Integrar el escenario Regression en el pipeline CI/CD para detectar degradaciones de rendimiento automáticamente en cada despliegue.

5. Conclusiones

5.1 Conclusiones generales

El proyecto Post Validator demostró que una estrategia de pruebas en pirámide, combinando pruebas unitarias, de integración, de sistema y de rendimiento, permite detectar defectos en múltiples capas que ningún nivel de prueba puede cubrir de forma aislada. Los 65 tests implementados forman una red de seguridad que protege las reglas de negocio, el contrato HTTP y el rendimiento del sistema de forma complementaria.

El cumplimiento del 100% de los SLOs de rendimiento y la cobertura global del 88% confirman que el sistema tiene una base sólida de calidad. Los 7 defectos identificados, lejos de ser un indicador negativo, demuestran la efectividad del proceso de pruebas: cada nivel encontró defectos que los otros no podían detectar.

5.2 Reflexión Final

El desarrollo de Post Validator nos permitió vivir de primera mano el valor de una estrategia de pruebas en pirámide aplicada de forma rigurosa. Cada nivel de prueba aportó una perspectiva distinta sobre la calidad del sistema y detectó defectos que los demás niveles no podían ver.

Las pruebas unitarias con TDD nos obligaron a pensar en los casos de error antes de escribir una sola línea de código, lo que resultó en un diseño más limpio y en la detección temprana del defecto de búsqueda case-insensitive. Las pruebas de integración y sistema revelaron que la seguridad de tipos de TypeScript opera en tiempo de compilación pero no en tiempo de ejecución, lo que hace indispensable la validación explícita de entradas en la capa HTTP. Y las pruebas de rendimiento demostraron que un sistema puede superar todas las pruebas funcionales y aun así presentar limitaciones estructurales que solo se manifiestan bajo carga sostenida.

Este proyecto nos convenció de que la calidad no es una etapa al final del desarrollo sino un proceso continuo que debe integrarse desde las primeras decisiones de diseño. En proyectos futuros aplicaremos TDD desde el inicio, definiremos los SLOs como requisitos no funcionales desde el día uno y automatizaremos la validación de rendimiento dentro del pipeline CI/CD.

6. Referencias

- Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing (3rd ed.). Wiley.

- Crispin, L., & Gregory, J. (2020). Agile Testing Condensed. Agile Testing Fellowship.
- Beck, K. (2003). Test-Driven Development: By Example. Addison-Wesley.
- Universidad de La Sabana. (2026). Guías de actividades U3, U4, U5 y U6. Curso Testing y Validación de Software.
- Grafana Labs. (2026). k6 Documentation. <https://k6.io/docs/>
- Jestjs.io. (2026). Jest Documentation. <https://jestjs.io/docs/>